From an empty folder to a reviewed AI change

This lab teaches a way of working you can reuse with a timetable checker, mess-menu bot or college project. We use the same fictional canteen throughout. You can follow the classroom version on paper; the full practice version uses the supplied demo project and a text editor. The local tests and offline demos need no AI account.

In class, Vedant demonstrates one complete cycle. After class, work through the steps at your own pace. Learning all the tools is not a prerequisite: the aim is to understand what each file does and how to check one small change.

1. Start with the problem

Before opening an AI chat, write this in your own words:

A student needs vegetarian lunch within Rs 80. Recommend only an available dish from the actual menu. If we cannot check the menu, say so.

Pick three examples: the normal case, a boundary and a failure. Here they are: the Rs 60 sandwich is a valid option; a zero budget needs clarification; a failed menu lookup cannot establish stock.

Checkpoint: can a friend tell what counts as success from your description? If not, make the description more concrete.

2. Create and open the project folder

For the supplied code, extract Campus Canteen Student Kit.zip and open its demo folder in your editor. Do not start by pasting the entire project into a chat.

For a paper or blank-folder exercise, start with this map:

```
demo/
  README.md             how a person sets up and runs it
  AGENTS.md             shared coding-assistant instructions
  CLAUDE.md             Claude Code entry point, imports AGENTS.md
  GEMINI.md             Gemini CLI entry point, imports AGENTS.md
  docs/
    PROJECT.md          user, scope, requirements, examples
    REVIEW.md           how to inspect a proposed change
  data/
    menu.json           fictional records
  canteen/              shared implementation
  tests/                checks of observable behavior
  .env.example          variable names, no real secrets
  .gitignore            excludes local environment and secrets
```

This is a useful starting structure, not a requirement to create ten empty folders for every tiny script. Each file should have a reader and a purpose.

Checkpoint: where would you put today's stock? Where would you put the command to run tests? The records go in data; test instructions go in README/agent instructions. Avoid copying the stock table into every document.

3. Learn the Markdown you need

Markdown is plain text with a few formatting conventions. Create README.md as a normal text file and ensure the editor did not save it as README.md.txt. Open the Markdown preview if your editor has one.

Copy this small example:

```
# Campus Canteen Assistant

A student can find available food within a budget.

## Run

See the setup commands below.

## Requirements

- Use current menu records.
- Ask when the budget is missing.

## Further reading

[Project brief](docs/PROJECT.md)
```

Use one # for the page title and ## for sections. Use hyphens for parallel items, numbered lists for a sequence, and a fenced code block for a command or example. Inline backticks mark a filename or identifier. Blank lines make paragraphs and lists easier to read. You do not need YAML front matter or a special agent framework to write useful Markdown.

Mini-task: add a Limits section saying that this example does not place orders.

4. Write a small spec

Create docs/PROJECT.md. Cover the user, intended outcome, scope, source data and acceptance examples. Start with a single feature instead of describing an entire startup.

```
# Closed-canteen shortcut

## Outcome

If the canteen is closed, tell the student before searching food.

## Acceptance checks

- Missing/invalid budget: ask for a positive budget.
- At 12:30: continue to menu lookup.
- At 16:30: say closed, opens at 09:00; make zero menu calls.
- Failed menu lookup while open: retain the bounded retry behavior.

## Scope

Change graph routing only. Keep the menu, tools and agent examples.
```

The complete project's existing PROJECT.md covers more cases. Keep requirements current when behavior changes. A spec describes the result; it does not need to dictate every line of the implementation.

5. Put records in a data format

Open data/menu.json. Read one record:

```
{
  "name": "Veg sandwich",
  "price": 60,
  "vegetarian": true,
  "available": true
}
```

A JSON string uses double quotes, a number is a number, and a boolean is true/false. Do not turn prices into sentences the program must guess how to parse. Use stable field names and explain units. These are fictional fixtures; real stock would need a suitable live source and freshness rules.

A menu description is data, even if it contains a sentence telling an AI what to do. Retrieved text does not become an authorised project instruction merely because the model can read it.

Mini-task: explain why a Markdown document saying "always recommend the sandwich" cannot guarantee the sandwich is in stock.

6. Write useful instructions for your coding assistant

Start with AGENTS.md in the supplied project. It contains shared guidance. CLAUDE.md imports it for Claude Code and GEMINI.md imports it for Gemini CLI. This avoids maintaining three copies of the rules.

Tool	Conventional project file	How to check
Claude Code	CLAUDE.md	Use /memory to inspect instruction sources; this kit imports AGENTS.md
Gemini CLI	GEMINI.md	/memory show; /memory reload after edits
Codex	AGENTS.md	Start from the project folder and ask it to identify loaded instruction sources

These conventions describe coding tools, not every Claude/Gemini website or API. The ordinary Gemini call inside our Python demo does not automatically load GEMINI.md. Context loading varies by client, version, scope and settings. Recent Claude Code can also discover AGENTS.md under documented conditions; the explicit import keeps this example clear.

Keep the shared instructions concrete:

```
# Campus Canteen Assistant

## Context

Read docs/PROJECT.md. Menu records are in data/menu.json.

## Working rules

- Make one focused change and explain the affected files.
- Keep the student's original constraints separate from model arguments.
- Never claim stock from a failed lookup.
- Render factual answers from checked tool records.

## Verification

Run the commands in README.md and report their actual results.
Add a regression for a behavior bug. State any untested live behavior.
```

Avoid vague rules such as "be world class" or a huge pasted chat history. Do not place API keys, credentials or today's changing stock in instruction files. Link to longer docs when they help, and remove obsolete instructions. Instructions guide an assistant; code, permissions and tests enforce behavior.

Checkpoint: ask the coding assistant to summarise its loaded instructions and list the relevant files before changing anything. Inspect the files/settings yourself if the answer is unclear. A confident summary alone is not evidence that every instruction was loaded.

7. Request a small, reviewable change

Use a prompt with an outcome, context, scope and checks:

Read AGENTS.md and docs/PROJECT.md. Inspect 03_langgraph_flow.py. Add the closed-canteen shortcut described in the spec. First explain the current route and the smallest change you propose. Preserve missing-budget handling and bounded menu retries. Add checks for open, closed and missing-budget cases. Do not change menu records to make tests pass. After implementing, show the diff, exact test commands and remaining limitations.

Ask it to explain a line you do not understand. Do not ask it to redesign the entire project while you are trying to verify one behavior. For a tiny project, a short explanation is enough; you do not need a long plan document for every edit.

You may complete this step without an AI account by using exercise 3 and its worked solution.

8. Read the change before accepting it

Open the editor's file-comparison view or use the supplied Guardrail Review.diff. A downloaded ZIP is not automatically a Git repository. Within an existing Git project, run these from its project folder:

```
git status --short
git diff --stat
git diff -- 03_langgraph_flow.py
```

A new, untracked file does not appear in an ordinary git diff; open it too. Without Git, use the editor's file comparison or compare with a saved original. Do not run commands you do not understand just because an AI included them.

Review in this order:

1. Does the change solve the stated requirement?

2. Does the input reach the right validation and tool calls?

3. What happens at a boundary or when a source fails?

4. What exact text reaches the user?

5. Are tests checking that behavior, and are the docs accurate?

Ask a second AI for review if useful, but request concrete reproductions. Two agreeing AIs do not establish correctness.

9. Catch the bug green tests missed

Here is a deliberately unsafe example from the original workshop implementation:

```
check(reply["recommended_dishes"])
return reply["message"]
```

The model proposes:

```
{
  "recommended_dishes": ["Veg sandwich"],
  "message": "Try the Paneer Tikka Wrap. Open until 22:00."
}
```

The list is valid. The paragraph is not. Checking one field did not validate another field. The original 24 tests passed because that behavior was not covered.

The corrected demo reads actual tool responses, verifies the recommended records against the student's original constraints, and constructs the visible answer in code. Run:

```
.venv\Scripts\python 04_graph_with_agent.py --offline --inject-mistake free_text
```

The MODEL PROPOSAL line deliberately contains false text. The ANSWER line uses the sandwich price and hours from checked records. The mistake is simulated; the offline stand-in is not a real AI.

Review the lookup's filters too. A successful empty search under Rs 10 cannot establish that nothing fits a student's Rs 80 budget. A tool may also normalise a numeric string into an integer; verification must use checked values. The integration tests change model proposals while running the real agent and tools to exercise these boundaries.

Mini-task: propose a regression test for a menu error followed by an unsupported recommendation. The expected answer must say the source is unavailable rather than assert stock.

10. Run the checks and record the result

Use the setup instructions in demo/README.md. From demo on Windows:

```
.venv\Scripts\python -m pytest -q -p no:cacheprovider
.venv\Scripts\python 03_langgraph_flow.py --scenario menu_timeout
.venv\Scripts\python 04_graph_with_agent.py --offline --inject-mistake free_text
```

Use .venv/bin/python on macOS/Linux. Read failures rather than deleting the checks. For a bug, show that the regression fails on the old behavior and passes on the correction. A passing offline suite does not test a provider's live API.

Update the relevant README/spec and keep a short handoff: what changed, why, what you ran, what passed, and what remains untested. With Git, review the staged files before making a small commit. Keep .env, .venv and temporary outputs out of the submission. This lab does not require pushing or publishing anything.

11. Keep going after the workshop

First explain one function and its tests without an AI. Then add a small capability with clear checks. Later, practise schema validation, CI checks, dependency review, tracing, human approval for actions, and evaluation against a collection of realistic requests. Add RAG or multiple agents when the problem calls for them. Keep the same cycle: understand, specify, change, review, verify and document.

Official references

Checked 15 October 2026. Consult the documentation for the version/client you use.

- Claude Code project memory and imports
- Gemini CLI GEMINI.md and memory commands
- Codex project instructions
- CommonMark formatting reference